CASE STUDY 01: ATTENDANCE RECOVERY CALCULATOR

1. Problem Statement:

Develop a Python program to calculate the additional consecutive classes a student must attend to reach a target attendance percentage.

2.Objectives:

- Compute current attendance percentage based on classes conducted and attended.

- Iteratively calculate required extra consecutive classes if the target is not met.

- Validate input data against realistic boundaries.

3.Assumptions:

- Classes conducted and attended are non-negative integers. - Target attendance is between 0 and 100 percent.

4. Algorithm:

1. Read total classes conducted, classes attended, and target percentage.

2. Validate inputs (conducted > 0, 0 <= attended <= conducted, 0 <= target <= 100).

3. Initialize extra class counter to 0.

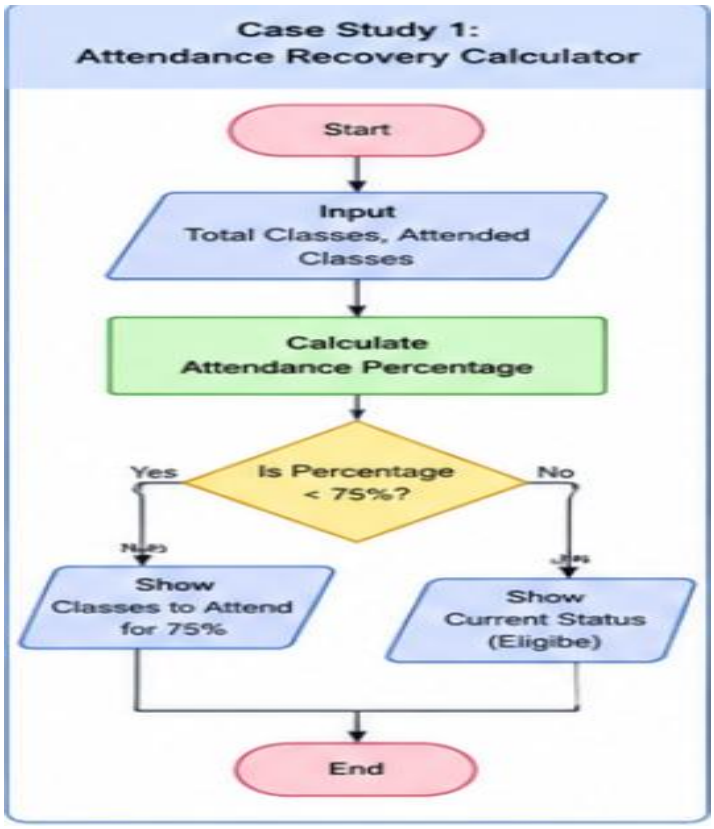
4. While current percentage < target: - Increment conducted by 1. - Increment attended by 1. - Increment extra counter by 1.

5. Print current attendance and extra classes required.

5.Pseudocode

```
WHILE (attended / conducted) * 100 < target:
    conducted = conducted + 1
    attended = attended + 1
    extra = extra + 1
END WHILE
```

5. Flowchart:



6. Python Source Code:

```
conducted = int(input("Enter total classes conducted: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))
    if conducted <= 0 or attended < 0 or attended > conducted:
        print("Invalid class details.")
   elif target < 0 or target > 100:
        print("Target must be between 0 and 100.")
  else:
      current = attended / conducted * 100
      extra = 0


   while current < target:
```

```
    conducted += 1
    attended += 1
    extra += 1
    current = attended / conducted * 100

print("Current attendance:", round(attended / conducted * 100, 2), "%")
if extra == 0:
    print("You have already reached the target.")
else:
    print("Additional consecutive classes required:", extra)
```

7. Test Cases & Sample Output

Inputs

Conducted = 50,

Attended = 35,

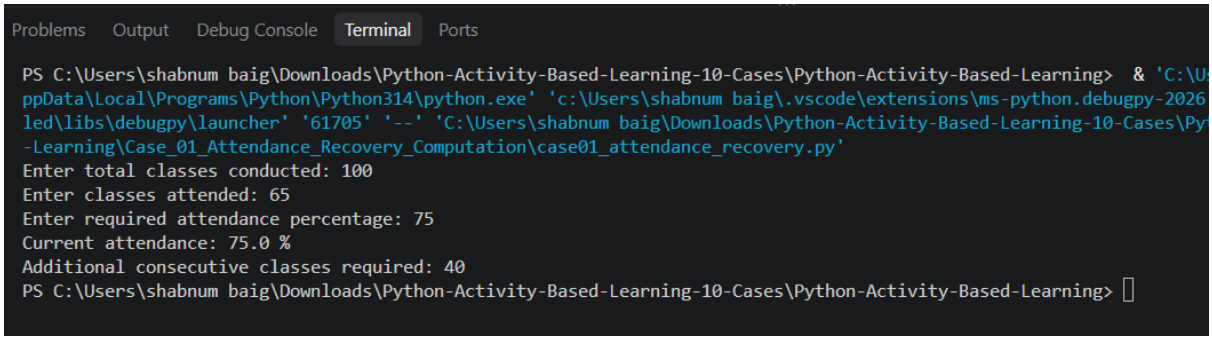
Target = 75%

Execution Output

Current attendance: 75.0 %
Additional consecutive classes required: 5

Current attendance: 95.0 %
You have already reached the target.

8.output screenshot:



Inputs

9. Conclusion

The program calculates required extra classes correctly using a while loop and handles invalid user inputs properly.

Case Study 2: ATM Transaction Simulator

1. Problem Statement & Objectives

Simulate an ATM machine that processes deposits and withdrawals while tracking valid and rejected transactions.

2. Assumptions & Scope

- Initial balance and transaction count ≥ 0.
- Supported transaction types: 'D' (Deposit), 'W' (Withdrawal).

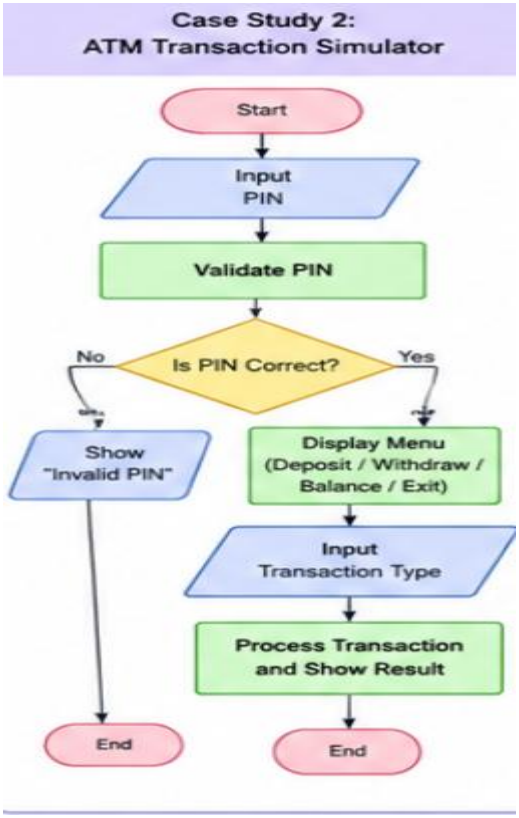
3. Algorithm

1. Input initial balance and number of transactions N.
2. Loop N times, taking transaction type and amount.
3. Validate transaction and update balance or increment rejected counter.
4. Display summary of balance, deposits, withdrawals, and rejected count.

4. Pseudocode

```
FOR i = 1 to N:
  IF type == 'D' AND amt > 0 THEN balance += amt; deposits++
  ELSE IF type == 'W' AND amt <= balance THEN balance -= amt; withdrawals++
  ELSE rejected++
END FOR
```

5. Flowchart



6. Python Source Code (case02_atm_simulation.py)

```
balance = float(input("Enter initial balance: "))

n = int(input("How many transactions? "))

deposits = 0

withdrawals = 0

rejected = 0


if balance < 0 or n < 0:

   print("Invalid starting balance or transaction count.")

else:

   for i in range(n):

      kind = input("Enter D for deposit or W for withdrawal: ").upper()

      amount = float(input("Enter amount: "))
```

```
    if amount <= 0:
        print("Invalid amount. Transaction rejected.")
        rejected += 1
    elif kind == "D":
        balance += amount
        deposits += 1
        print("Deposit successful.")
    elif kind == "W":
        if amount <= balance:
            balance -= amount
            withdrawals += 1
            print("Withdrawal successful.")
        else:
            print("Insufficient balance. Transaction rejected.")
            rejected += 1
    else:
        print("Invalid transaction type.")
        rejected += 1

print("\nTransaction Summary")
print("Final balance:", round(balance, 2))
print("Successful deposits:", deposits)
print("Successful withdrawals:", withdrawals)
print("Rejected transactions:", rejected)
```

7. Test Cases & Sample Output

Inputs

Initial Bal: 5000, 2 Transactions: D
1000, W 6000

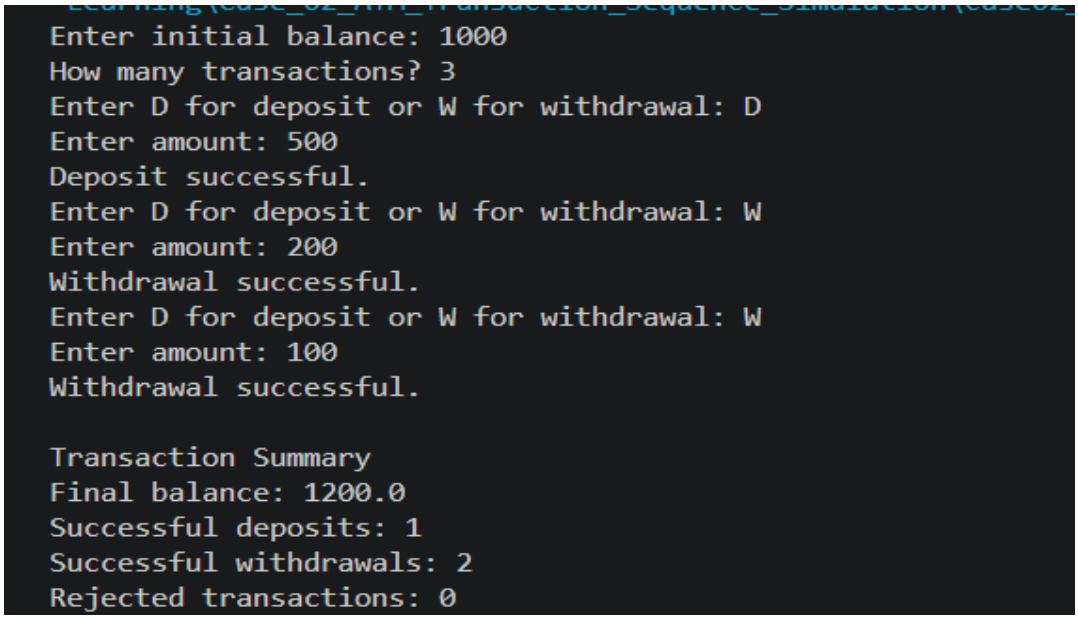
Initial Bal: 2000, 1 Transaction: W 500

Execution Output

Deposit successful.
Insufficient balance. Transaction rejected.
Final balance: 6000.0 | Successful deposits: 1 | Successful withdrawals: 0 | Rejected: 1

Withdrawal successful.
Final balance: 1500.0 | Successful deposits: 0 | Successful withdrawals: 1 | Rejected: 0

8.Output screenshot:



8. Conclusion

Simulates real-world ATM logic accurately, enforcing balance rules and logging rejected transactions.

Case Study 3: Academic Performance Trend Analyzer

1. Problem Statement & Objectives

Compare student performance across two exams and classify the trend as Improved, Declined, or No Change.

2. Assumptions & Scope

- Subjects: Python, Maths, English.
- Marks must be strictly between 0 and 100 for each exam.

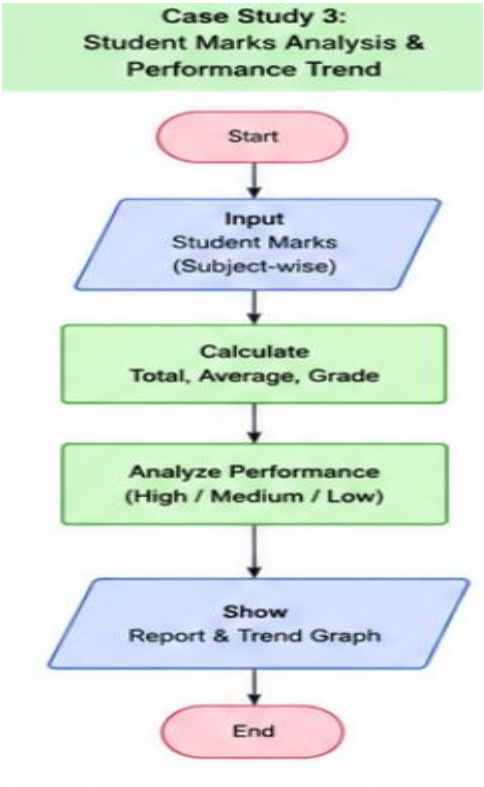
3. Algorithm

1. Loop through subject list, prompting Exam 1 and Exam 2 marks.
2. Validate mark ranges; terminate if invalid.
3. Compare exam scores per subject and determine performance status.
4. Output subject-wise status and total mark difference.

4.Pseudocode

```
FOR subject IN subjects:
  IF mark1 < 0 OR mark2 > 100 THEN BREAK
  exam1.append(mark1); exam2.append(mark2)
ELSE:
  COMPUTE score differences and total overall change
```

5. Flowchart



6. Python Source Code (case03_performance_trend.py)

```
subjects = ["Python", "Maths", "English"]

exam1 = []

exam2 = []


for subject in subjects:

  first = float(input("Enter " + subject + " marks in Exam 1: "))

  second = float(input("Enter " + subject + " marks in Exam 2: "))

  if first < 0 or first > 100 or second < 0 or second > 100:

    print("Marks must be between 0 and 100. Please restart and enter valid marks.")

    break

  exam1.append(first)

  exam2.append(second)

else:
```

```
print("\nPerformance Trend")
for i in range(len(subjects)):
    if exam2[i] > exam1[i]:
        status = "Improved"
    elif exam2[i] < exam1[i]:
        status = "Declined"
    else:
        status = "No change"
    print(subjects[i], ":", status, "(change:", exam2[i] - exam1[i], ")")

print("Exam 1 total:", sum(exam1))
print("Exam 2 total:", sum(exam2))
print("Overall change:", sum(exam2) - sum(exam1))
```

7. Test Cases & Sample Output

Inputs (Exam 1, Exam 2)

Python (80, 85), Maths (90, 85), English (70, 75)

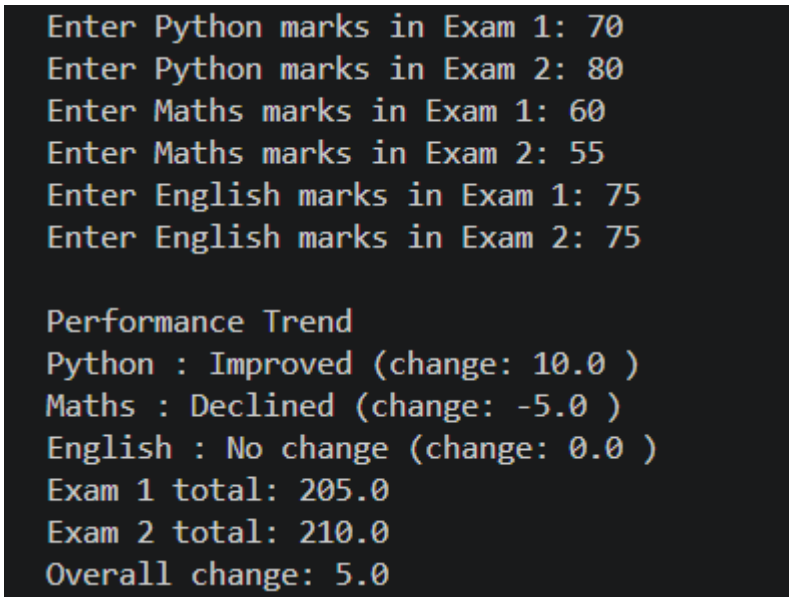
Execution Output

Python : Improved (change: 5.0)
Maths : Declined (change: -5.0)
English : Improved (change: 5.0)
Exam 1 total: 240.0 | Exam 2 total: 245.0 | Overall change: 5.0

Marks must be between 0 and 100. Please restart.

Inputs (Exam 1, Exam 2)

8.output screenshot:



9. Conclusion

Demonstrates parallel list processing and Python's for-else construct for clean data reporting

Case Study 4: Password Validation System

1. Problem Statement & Objectives

Validate a password string against complexity rules and list all specific rule violations.

2. Assumptions & Scope

- Min 8 chars, 1 uppercase, 1 lowercase, 1 digit, 1 special char.
- Special characters are verified using non-alphanumeric check.

3. Algorithm

1. Read input password string.
2. Evaluate length, uppercase, lowercase, digit, and special characters.
3. Store rule failure messages in an error list.
4. If error list is empty, output success; otherwise list all errors.

4. Pseudocode

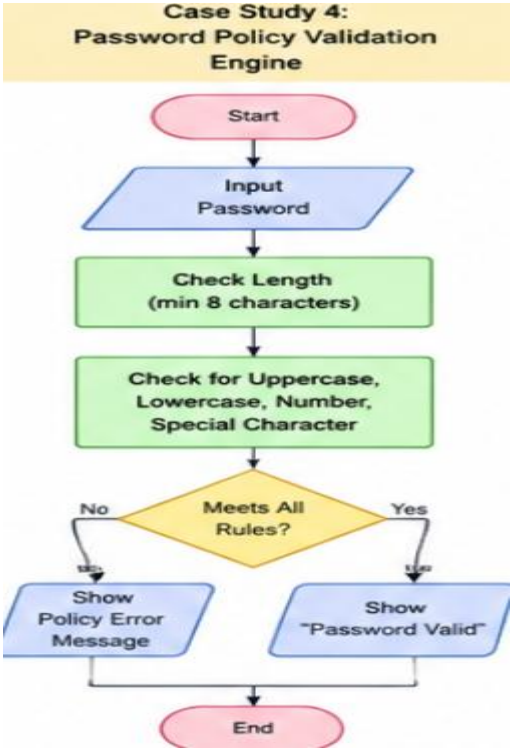
IF len(pwd) < 8 THEN errors.append("Use at least 8 characters.")

IF NOT has_upper THEN errors.append("Add at least one uppercase letter.")

IF NOT has_special THEN errors.append("Add at least one special character.")

IF len(errors) == 0 THEN Print "Success" ELSE Print errors

5. Flowchart



6. Python Source Code (case04_password_validation.py)

```
password = input("Enter a sample password to check: ")

errors = []


if len(password) < 8:
    errors.append("Use at least 8 characters.")
if not any(ch.isupper() for ch in password):
    errors.append("Add at least one uppercase letter.")
if not any(ch.islower() for ch in password):
```

```
    errors.append("Add at least one lowercase letter.")
if not any(ch.isdigit() for ch in password):
    errors.append("Add at least one digit.")
if not any(not ch.isalnum() for ch in password):
    errors.append("Add at least one special character.")


if len(errors) == 0:
    print("Password meets all sample rules.")
else:
    print("Password does not meet the sample rules:")
    for error in errors:
        print("-", error)
```

7. Test Cases & Sample Output

Input Password

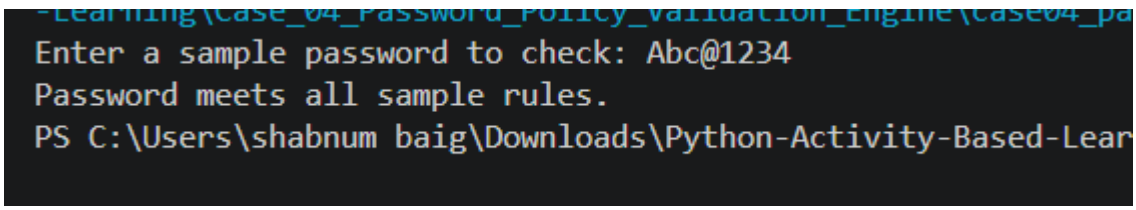
Pass@1234

Execution Output

Password meets all sample rules.

Password does not meet the sample rules:
- Use at least 8 characters.
- Add at least one uppercase letter.
- Add at least one digit.
- Add at least one special character.

8.outout screenshot:



9. Conclusion

Uses string methods and generators effectively to validate user input against security policies.

Case Study 5: MCQ Answer Evaluator

1. Problem Statement & Objectives

Evaluate student MCQ responses against an answer key and compute total score with negative marking.

2. Assumptions & Scope

- Answer key: ['A', 'C', 'B', 'D', 'A'].
- Blank response is '-'. Scoring: Correct +1, Incorrect -0.25, Blank 0.

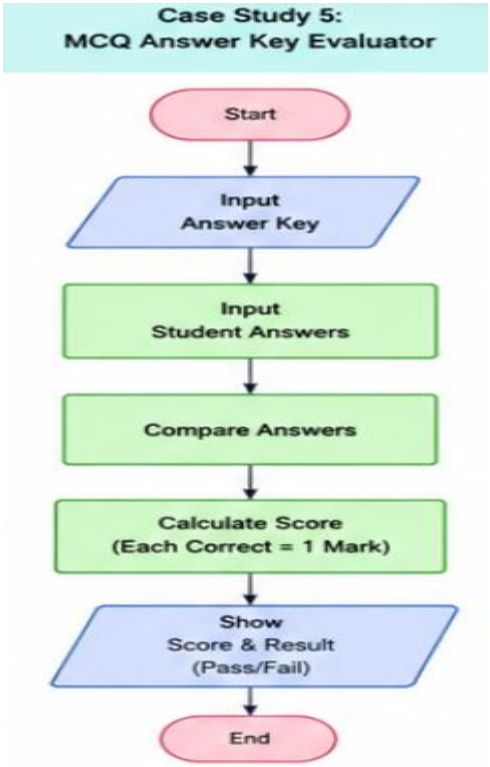
3. Algorithm

1. Input student answers as space-separated values.
2. Verify answer count matches key length.
3. Loop through answers: count correct, incorrect, and unanswered.
4. Calculate final score = correct - (incorrect * 0.25).

4. Pseudocode

```
FOR i = 0 to 4:
        IF ans[i] == '-' THEN unanswered++
   ELSE IF ans[i] == key[i] THEN correct++
   ELSE incorrect++
score = correct - (incorrect * 0.25)
```

5. Flowchart



6. Python Source Code (case05_answer_evaluation.py)

```
answer_key = ["A", "C", "B", "D", "A"]

answers = input("Enter 5 answers separated by spaces (use - for blank): ").upper().split()


if len(answers) != len(answer_key):
   print("Please enter exactly", len(answer_key), "answers.")
else:
   correct = 0
   incorrect = 0
   unanswered = 0


   for i in range(len(answer_key)):
```

```
        if answers[i] == "-":
            unanswered += 1
        elif answers[i] == answer_key[i]:
            correct += 1
        else:
            incorrect += 1

    score = correct - incorrect * 0.25
    print("Correct answers:", correct)
    print("Incorrect answers:", incorrect)
    print("Unanswered questions:", unanswered)
    print("Final score:", round(score, 2), "out of", len(answer_key))
```

7. Test Cases & Sample Output

Input Answers

A C B - B

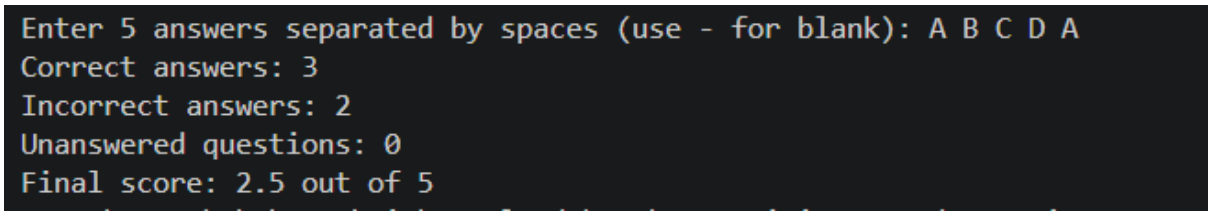
A C B D A

Execution Output

Correct answers: 3
Incorrect answers: 1
Unanswered questions: 1
Final score: 2.75 out of 5

Correct answers: 5
Incorrect answers: 0
Unanswered questions: 0
Final score: 5.0 out of 5

8.output screenshot:



9. Conclusion

Accurately evaluates student performance using list comparisons and floating-point arithmetic.

Case Study 6: Marks Matrix Analysis

1. Problem Statement & Objectives

Process a 2D matrix of student marks to calculate individual totals, subject averages, and top performer.

2. Assumptions & Scope

- 3 Students (Asha, Ravi, Sara), 3 Subjects (Python, Maths, English).
- Data organized in 2D matrix list `marks[student][subject]`.

3. Algorithm

1. Sum rows for individual student totals and averages.
2. Iterate columns to calculate overall subject averages.
3. Identify index of maximum total score to find top performer.
4. Display all computed metrics clearly.

4. Pseudocode

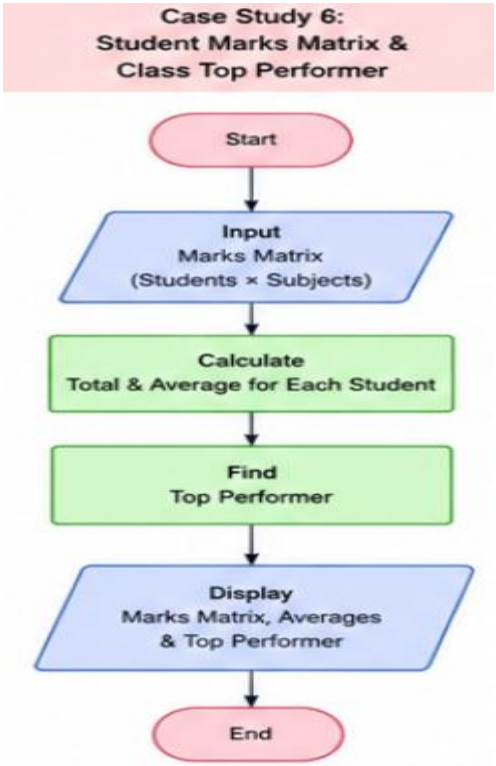
FOR row IN marks: totals.append(sum(row))

FOR col j = 0 to 2:

  column_total = sum(marks[i][j] for i in 0..2)

top_performer = students[totals.index(max(totals))]

5. Flowchart



6. Python Source Code (case06_marks_matrix.py)

```
students = ["Asha", "Ravi", "Sara"]
subjects = ["Python", "Maths", "English"]
marks = [
  [85, 78, 90],
  [72, 88, 80],
  [92, 91, 89]
]

totals = []
for i in range(len(marks)):
  total = sum(marks[i])
  totals.append(total)
  print(students[i], "total:", total, "average:", round(total / len(subjects), 2))
```

```
print("\nSubject averages:")
for j in range(len(subjects)):
    column_total = 0
    for i in range(len(marks)):
        column_total += marks[i][j]
    print(subjects[j], ":", round(column_total / len(students), 2))

top_index = totals.index(max(totals))
print("Top performer:", students[top_index], "with", totals[top_index], "marks")
```

7. Test Cases & Sample Output

Metric Evaluated

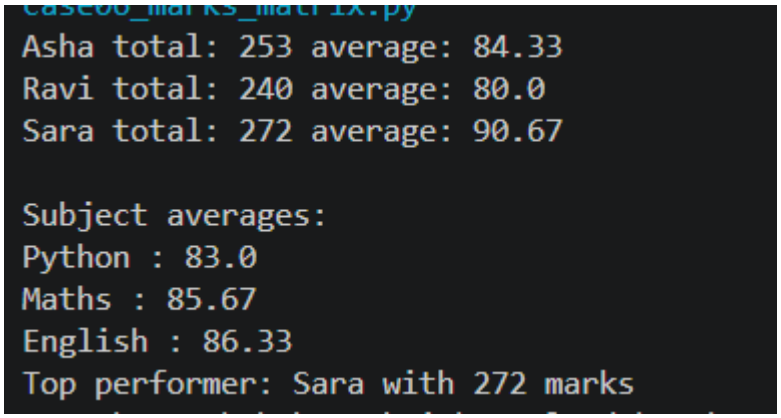
Student Totals & Averages

Execution Output

Asha total: 253 average: 84.33
Ravi total: 240 average: 80.0
Sara total: 272 average: 90.67

Subject averages: Python : 83.0 | Maths : 85.67 | English : 86.33
Top performer: Sara with 272 marks

8.output screenshot:



8. Conclusion

Demonstrates 2D list manipulation and row/column nested loop iterations.

Case Study 7: Product Catalogue Inventory Audit

1. Problem Statement & Objectives

Audit product tuples in a catalogue list, compute inventory valuations, and trigger restocking alerts.

2. Assumptions & Scope

- Tuple structure: `(product_id, price, quantity)`.
- Low stock replenishment alert triggered if quantity < 5.

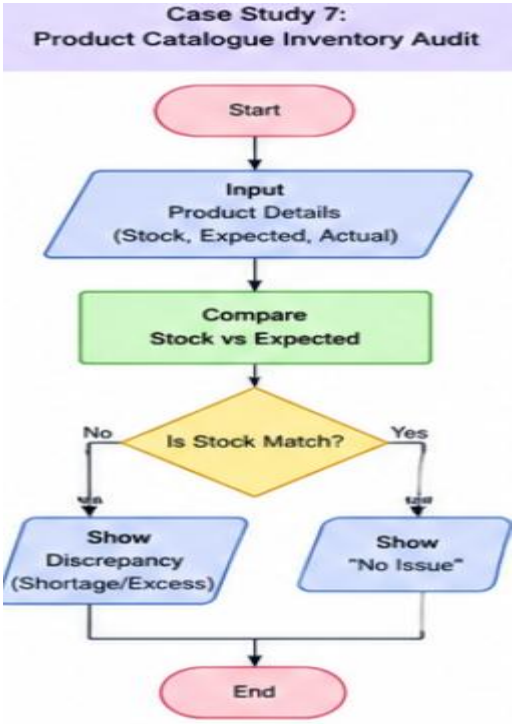
3. Algorithm

1. Loop through product tuples in catalogue list.
2. Unpack tuple into `product_id, price, quantity`.
3. Validate price and quantity values.
4. Calculate inventory value; alert if stock < 5; compute total.

4. Pseudocode

```
FOR (id, price, qty) IN products:
  IF price < 0 OR qty < 0 THEN Print "Invalid record"
  ELSE val = price * qty; total_value += val
  IF qty < 5 THEN Print "Replenishment needed"
```

5. Flowchart



6. Python Source Code (case07_product_catalogue.py)

```
products = [
    ("P101", 50.0, 10),
    ("P102", 120.0, 3),
    ("P103", 75.0, 0),
    ("P104", 40.0, 8)
]


total_value = 0
print("Product Catalogue Audit")


for product in products:
    product_id, price, quantity = product


    if price < 0 or quantity < 0:
```

```
        print(product_id, "- Invalid record")
    else:
        value = price * quantity
        total_value += value
        print(product_id, "inventory value:", value)
        if quantity < 5:
            print("  Replenishment needed; current stock:", quantity)

print("Total inventory value:", round(total_value, 2))
```

7. Test Cases & Sample Output

Product Sample

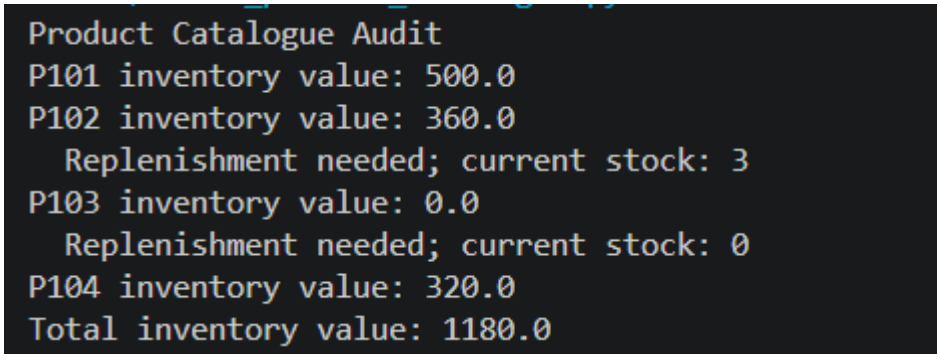
Catalogue Record Check

Execution Output

P101 inventory value: 500.0
P102 inventory value: 360.0
Replenishment needed; current stock: 3
P103 inventory value: 0.0
Replenishment needed; current stock: 0
P104 inventory value: 320.0

Total inventory value: 1180.0

8.output screenshot:



9. Conclusion

Uses tuple unpacking efficiently to process structured records and perform data audits.

Case Study 8: Canteen Food Wastage Monitor

1. Problem Statement & Objectives

Monitor daily meal preparation and consumption to calculate food wastage percentage and highlight excess waste.

2. Assumptions & Scope

- 5 days data (Mon-Fri).
- Served meals ≤ prepared meals. Warning threshold = 10%.

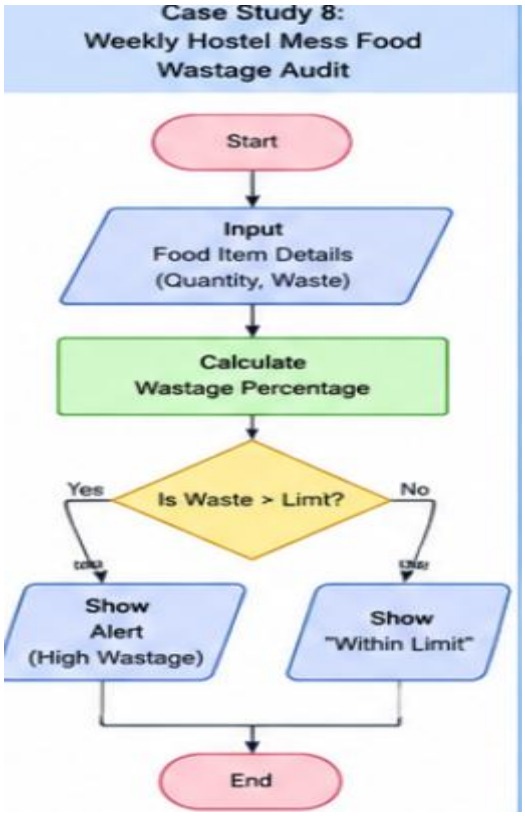
3. Algorithm

1. Loop through day indices 0 to 4.
2. Compute leftovers = prepared - served.
3. Calculate wastage percentage = (leftovers / prepared) * 100.
4. Output results and trigger warning if wastage % > threshold.

4. Pseudocode

```
FOR i = 0 to len(days)-1:
  leftovers = prepared_meals[i] - served_meals[i]
  waste_percent = (leftovers / prepared_meals[i]) * 100
  IF waste_percent > 10 THEN Print "Warning: Wastage exceeded"
```

5. Flowchart



6. Python Source Code (case08_food_waste.py)

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]

prepared_meals = [100, 120, 90, 110, 100]

served_meals = [92, 100, 85, 90, 70]

threshold = 10


for i in range(len(days)):
    prepared = prepared_meals[i]
    served = served_meals[i]


    if prepared <= 0 or served < 0 or served > prepared:
        print(days[i], "- Invalid meal counts")
    else:
        leftovers = prepared - served
```

```
    waste_percent = leftovers / prepared * 100
    print(days[i], "leftovers:", leftovers,
        "wastage:", round(waste_percent, 2), "%")
    if waste_percent > threshold:
       print("  Warning: Wastage exceeded", threshold, "%")
```

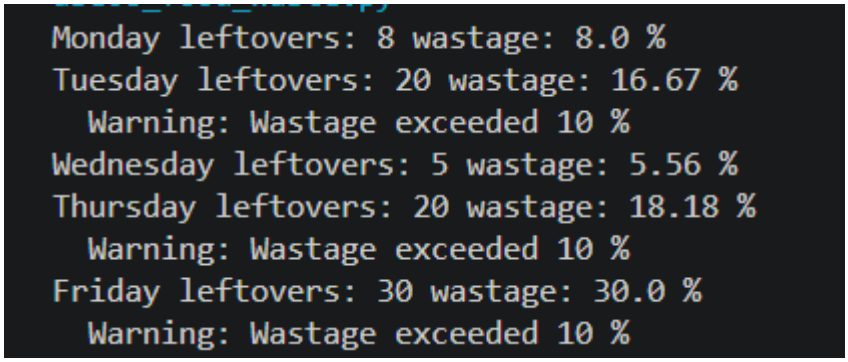
7. Test Cases & Sample Output

Day (Prepared/Served)

Execution Output

Monday leftovers: 8 wastage: 8.0 %
Tuesday leftovers: 20 wastage: 16.67 %
Warning: Wastage exceeded 10 %
Wednesday leftovers: 5 wastage: 5.56 %
Thursday leftovers: 20 wastage: 18.18 %
Warning: Wastage exceeded 10 %
Friday leftovers: 30 wastage: 30.0 %
Warning: Wastage exceeded 10 %

8.output screenshot:



9. Conclusion

Tracks operational metrics using parallel list indexing and raises conditional alerts.

Case Study 9: Canteen Billing & Stock Manager

1. Problem Statement & Objectives

Process item orders, update inventory stock dynamically, and calculate total daily sales.

2. Assumptions & Scope

- Items: Sandwich (Rs. 50), Samosa (Rs. 15), Juice (Rs. 25).
- Orders exceeding available stock are rejected.

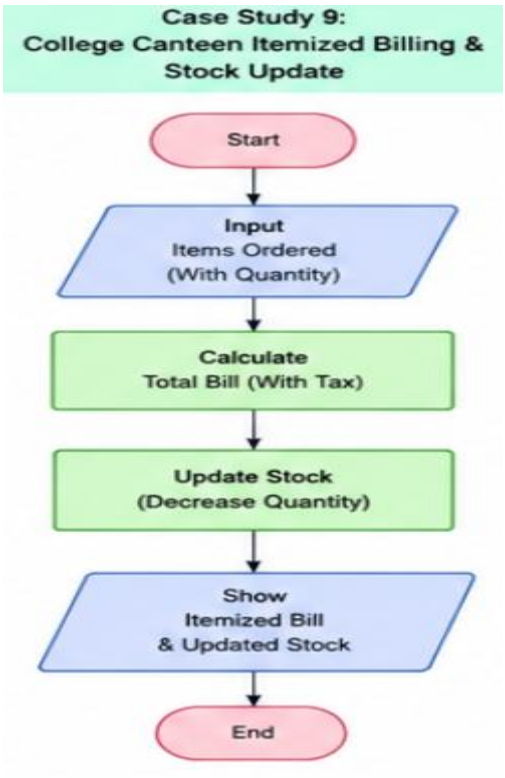
3. Algorithm

1. Display menu with item prices and current stock.
2. Prompt total orders count; loop to get item number and quantity.
3. Validate choice and stock level; deduct stock and compute bill.
4. Display final total daily sales and remaining inventory.

4. Pseudocode

```
FOR i = 1 to orders:
  IF valid_item AND quantity <= stock[idx]:
    stock[idx] -= quantity
    total_sales += prices[idx] * quantity
  ELSE: Print "Order rejected"
```

5. Flowchart



6. Python Source Code (case09_canteen_stock.py)

```
items = ["Sandwich", "Samosa", "Juice"]
prices = [50, 15, 25]
stock = [10, 20, 12]
total_sales = 0

print("Canteen Menu")
for i in range(len(items)):
    print(i + 1, items[i], "- Rs.", prices[i], "| Stock:", stock[i])

orders = int(input("Enter number of orders: "))

for i in range(orders):
    item_number = int(input("enter item number(1-3):"))
    quantity = int(input("Enter quantity: "))

    if item_number < 1 or item_number > len(items) or quantity <= 0:
        print("Invalid order.")
    else:
        index = item_number - 1
        if quantity > stock[index]:
            print("Not enough stock. Order rejected.")
        else:
            bill = prices[index] * quantity
            stock[index] -= quantity
            total_sales += bill
            print("Order accepted. Bill: Rs.", bill)
```

```
print("\nDaily Sales: Rs.", total_sales)

print("Remaining Stock:")

for i in range(len(items)):

    print(items[i], ":", stock[i])
```

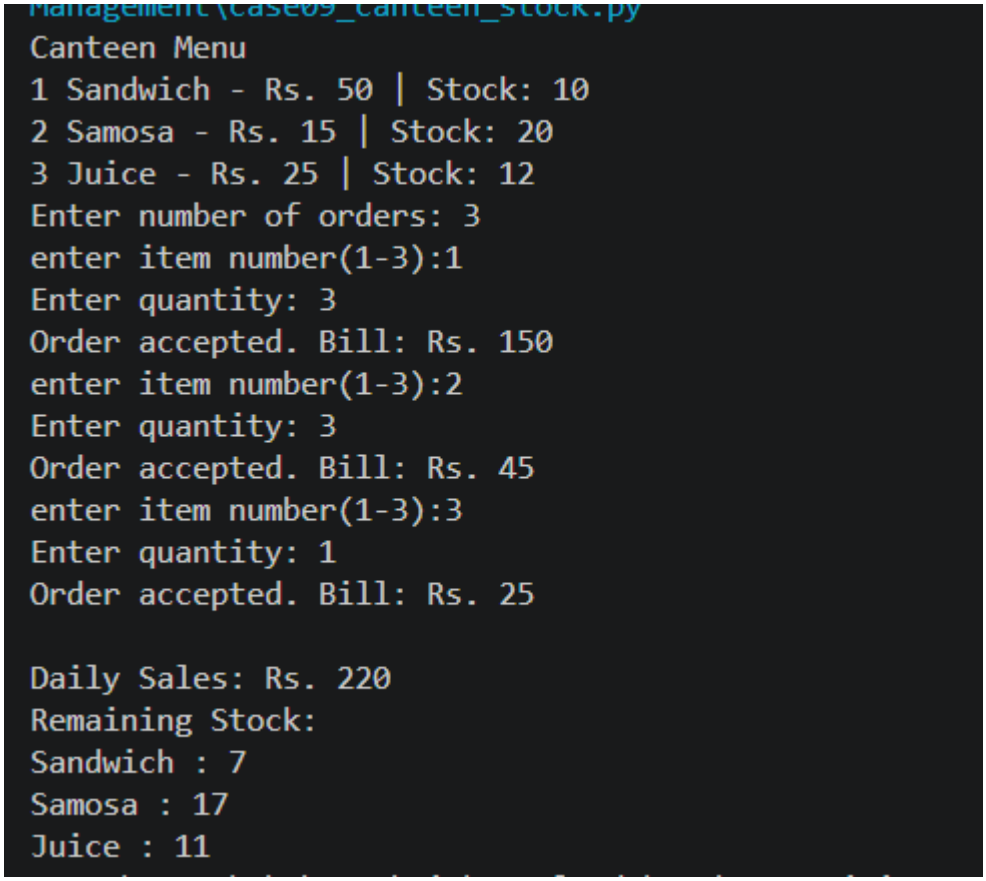
7. Test Cases & Sample Output

Order Input

Execution Output

Order accepted. Bill: Rs. 75
Not enough stock. Order rejected.
Daily Sales: Rs. 75
Remaining Stock: Sandwich : 10 | Samosa : 15 | Juice : 12

8.output screenshot:



8. Conclusion

Demonstrates list index mapping and interactive state updates for retail applications.

Case Study 10: Hollow Diamond Pattern Generator

1. Problem Statement & Objectives

Generate a symmetrical hollow diamond pattern using star (`*`) characters based on input size n.

2. Assumptions & Scope

- Diamond size n bounded: 2 ≤ n ≤ 20.
- Uses formula `2 * i - 3` for inside spaces calculation.

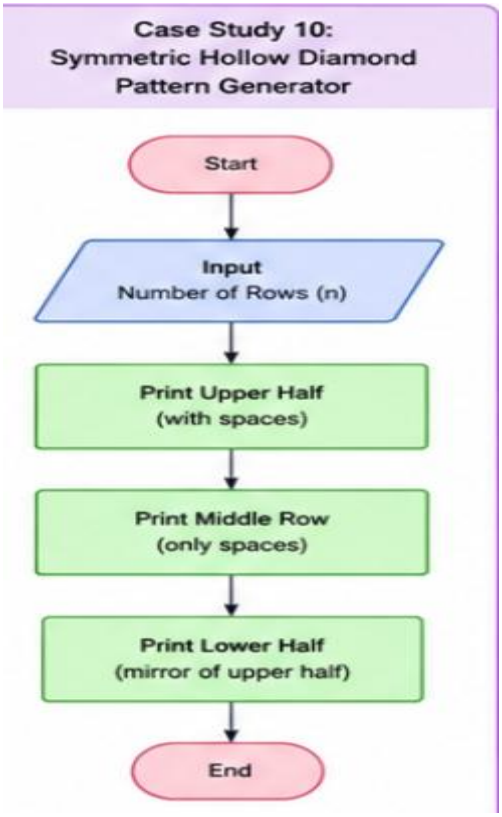
3. Algorithm

1. Input diamond size n; validate range [2, 20].
2. Top half: loop i from 1 to n; print leading spaces and stars.
3. Bottom half: loop i from n-1 down to 1; print corresponding inverted lines.

4. Pseudocode

```
FOR i = 1 to n:
  IF i == 1 THEN Print spaces + "*"
  ELSE Print spaces + "*" + inside_spaces + "*"
REPEAT inverted loop for i = n-1 DOWNTO 1
```

5. Flowchart



6. Python Source Code (case10_numeric_pattern.py)

```
n = int(input("Enter diamond size (2-20): "))

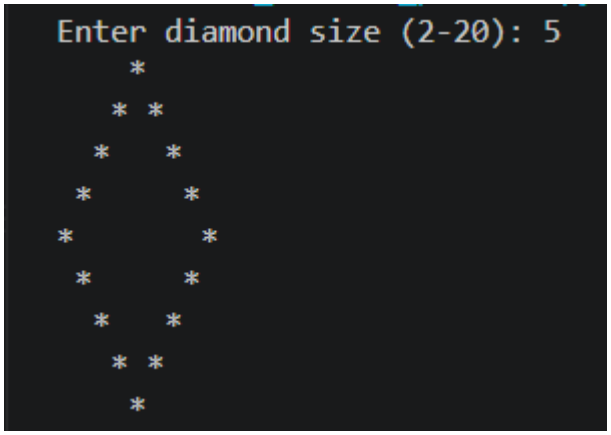

if n < 2 or n > 20:
   print("Please enter a size between 2 and 20.")
else:
   for i in range(1, n + 1):
      spaces = n - i
      if i == 1:
         print(" " * spaces + "*")
      else:
         inside_spaces = 2 * i - 3
         print(" " * spaces + "*" + " " * inside_spaces + "*")
```

```
for i in range(n - 1, 0, -1):
    spaces = n - i
    if i == 1:
        print(" " * spaces + "*")
    else:
        inside_spaces = 2 * i - 3
        print(" " * spaces + "*" + " " * inside_spaces + "*")
```

7. Test Cases & Sample Output (n = 4)

```
   *
  * *
 *   *
*     *
 *   *
  * *
   *
```

8.output screenshot:



9. Conclusion

Demonstrates pattern formatting, nested loop ranges, and string repetition in Python.